# IN1010 Eksamen — Cheatsheet

## ITERATOR

*Klassen implements Iterable<T>, indre klasse implements Iterator<T>*

```
class MinKlasse implements Iterable<T> {
    class MinIterator implements Iterator<T> {
        Node current;
        MinIterator() { current = start; }
        public boolean hasNext() {
            // Standard:
            return current != null;
            // Filtrer null-noder (V2025):
            // while (current != null && current.data == null)
            // current = current.neste;
            // return current != null;
        }
        public T next() {
            T data = current.data;
            current = current.neste;
            return data;
        }
    }
    public Iterator<T> iterator() { return new MinIterator(); }
}
```

---

## REKURSJON

*Basecase: siste node (current.neste == null) eller tom liste (current == null)*

```
public int metode() {
    if (foerste == null) return 0; // tom liste
    return metode(foerste);
}
private int metode(Node current) {
    if (current.neste == null) return current.verdi; // basecase
    return kombiner(current.verdi, metode(current.neste));
}
// Tell: 1 + metode(current.neste)
// Sum: current.verdi + metode(current.neste)
// Maks: Math.max(current.verdi, metode(current.neste))
```

---

## TRÅDER — MONITOR

*signalAll() i try (ikke finally) | await() i while (ikke if) | unlock() i finally*

```
class MinMonitor {
    ReentrantLock laas = new ReentrantLock();
    Condition venteBetingelse = laas.newCondition();
    ArrayList<T> liste = new ArrayList<>();
    void leggTil(T element) {
        laas.lock();
        try {
            liste.add(element);
            venteBetingelse.signalAll();
        } finally {
            laas.unlock();
        }
    }
    T hentNeste() {
        laas.lock();
        try {
            while (liste.isEmpty()) // ALLTID while, ikke if
                venteBetingelse.await();
```

```
            return liste.remove(0);
        } catch (InterruptedException e) { return null; }
        } finally { laas.unlock(); }
    }
}
```

## TRÅDER — TRÅDKLASSE

```
class MinTraad implements Runnable {
    MinMonitor monitor;
    T data;
    MinTraad(MinMonitor monitor, T data) {
        this.monitor = monitor; this.data = data;
    }
    public void run() {
        monitor.leggTil(data); // eller: monitor.hentNeste()
    }
}
```

---

## GUI — BOILERPLATE

*import javax.swing.*; import java.awt.event.*;*

```
JFrame vindu = new JFrame("Tittel");
vindu.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
JPanel panel = new JPanel(); vindu.add(panel);
panel.add(new JLabel("Tekst"));
JTextField felt = new JTextField(10); panel.add(felt);
JComboBox<String> valg = new JComboBox<>(); valg.addItem("Alt"); panel.add(valg);
JCheckBox boks = new JCheckBox("Valg"); panel.add(boks);
class Handler implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        String s = felt.getText();
        int n = Integer.parseInt(felt.getText());
        String v = (String) valg.getSelectedItem();
        boolean b = boks.isSelected();
        vindu.setVisible(false); // lukk vindu
    }
}
JButton ok = new JButton("OK");
ok.addActionListener(new Handler()); panel.add(ok);
vindu.pack(); vindu.setVisible(true);
```

---

## LENKELISTE — HUSKELISTE

*Sjekk alltid: tom liste | foerste element | siste element | midten*

```
// Opprett n noder
Node siste = null;
for (int i = 0; i < n; i++) {
    Node ny = new Node();
    if (start == null) { start = ny; siste = ny; }
    else { siste.neste = ny; siste = ny; }
}
// Soek
while (peker != null) {
    if (peker.id.equals(id)) return true;
    peker = peker.neste;
} return false;
// taUt
if (foerste == null) return null;
if (foerste.id.equals(id)) { foerste = foerste.neste; return fjernet; }
while (peker != null) {
    if (peker.neste != null && peker.neste.id.equals(id)) {
        fjernet = peker.neste; peker.neste = peker.neste.neste; return fjernet;
```

```
    }
    peker = peker.neste;
}
```